Le chiffrement de César

Le chiffrement de César est une méthode de cryptographie utilisée par César pour rendre certaines de ses correspondances privées secrètes. Ce dernier consiste simplement à prendre un message et à remplacer chaque lettre du message clair (non chiffré) par une lettre à distance fixe, dans l'ordre de l'alphabet. Ainsi, le mot CESAR chiffré à une distance de 3 devient FHVDU. Si jamais on dépasse la 26ème lettre de l'alphabet (Z), alors on continue à partir de A de manière cyclique. Ainsi, la lettre W décalée d'une distance de 9 donne F (WXYZABCDEF). Ce mini-projet dans sa version minimale consiste à implémenter en Python un chiffreur–déchiffreur de César, pour des messages écrits en majuscules, en omettant les caractères spéciaux (parties 1, 2 et 3). Plusieurs extensions sont proposées en partie 4, mais vous pouvez aussi implémenter les vôtres, après validation avec un enseignant.

1 Phase préliminaire : table ASCII et Python

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	”	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	’	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

En informatique, les caractères de nos claviers sont encodés. Le standard ASCII code 128 caractères incluant principaux caractères spéciaux et ponctuations, lettres en majuscules et minuscules, et chiffres. Ainsi, pour indiquer à l'ordinateur un caractère en particulier (« Char »), on va utiliser le code ASCII associé à ce dernier (« Dec »). Cela va nous être utile pour calculer les décalages entre lettres.

En Python, les deux fonctions qui permettent de passer de l'encodage au caractère sont `ord` et `chr`. Ainsi, `ord('A')` retournera la valeur 65, alors que `chr(47)` retournera /.

Afin de vous familiariser avec ces fonctions, commencez par écrire en Python la chaine de caracteres `"CESAR"` en utilisant uniquement la fonction `chr`. Inversement, donnez la séquence de codes ASCII qui représente le mot `"BONJOUR"`.

2 Fonction de chiffrement

Pour faire un chiffrement de César, on peut exploiter la table ASCII qui respecte l'ordre alphabétique. En particulier, on va se concentrer sur les lettres majuscules de la table, qui vont du code ASCII 65 au code ASCII 90.

Vous allez écrire une fonction `chiffrement_cesar` prenant en entrée le message en clair et la valeur de décalage. Il faut ensuite itérer sur tous les caractères du message (pour rappel, une chaîne de caractère est une liste sur laquelle vous pouvez itérer avec une boucle `for`), faire le décalage, et ajouter la nouvelle lettre dans une variable qui représente le message chiffré, que la fonction retournera.

Ainsi, pour chaque caractère du message en clair, les étapes à suivre sont les suivantes : (1) Regarder si le caractère est une lettre. Si ce n'est pas le cas, alors on ajoute le caractère du

message tel qu'il est dans le message chiffré. Sinon, voir les étapes qui suivent.[1]. (2) Obtenir le code ASCII de la lettre originale. (3) Y soustraire le code ASCII de la première lettre `'A'`, de sorte à avoir une valeur dans $[\![0, 25]\!]$. (4) Y ajouter le décalage. (5) Faire le modulo 26 de la valeur calculée jusque là pour résoudre les situations où on dépasse la lettre `'Z'`[2]. (6) Ajouter ensuite le code ASCII de la première lettre `'A'`, de sorte à avoir le code ASCII de la lettre décalée. (7) Transformer le code ASCII en un caractère, et l'ajouter au message chiffré. . Utiliser les exemples calculés à la main en partie 1 pour vérifier le fonctionnement du chiffrement.

3 Fonction de déchiffrement

Pour le déchiffrement, on va suivre le même processus, mais au lieu d'aller vers la droite dans l'alphabet pour le décalage, on va reculer, et donc aller vers la gauche. Écrire une fonction `dechiffrement_cesar` prenant prenant en entrée le message chiffré et la valeur de décalage. La fonction retournera le message en clair. Vous suivrez le procédé décrit dans la partie 2, excepté que (4) devient : Y soustraire le décalage. Faire les mêmes tests que précédemment, mais cette fois-ci en renseignant en paramètre les messages cryptés.

4 Propositions d'extensions

Voici quelques propositions d'extensions si vous allez au-delà de la version minimale. La première extension est obligatoire (elle est mineure), et vous pouvez ensuite choisir l'une des extensions qui suivent, ou bien implémenter la vôtre après l'avoir faite valider :

1. Ajouter une couche supplémentaire à votre code afin de faire utiliser le chiffreur–déchiffreur par un utilisateur. Utilisez la fonction `input`[3] pour récupérer le message, le décalage et s'il souhaite chiffrer ou déchiffrer son message. Faites ensuite appel à la fonction adéquate en fonction du choix (`chiffrement_cesar` ou `dechiffrement_cesar`), et retourner le message crypté ou decrypté. Compléter le code pour gérer les minuscules.
2. La méthode de chiffrement de César peut-être facilement cassée en testant les 26 décalages possibles et en regardant lequel des 26 messages est compréhensible. Cette extension consiste donc à adapter la fonction de déchiffrement en lui enlevant le paramètre de décalage, et à écrire avec des `print` les 26 messages possibles.
3. L'extension ci-dessus montre la fragilité du chiffrement de César. Un autre chiffrement plus abouti est le chiffre de Vigenère[4]. Au lieu de choisir un décalage fixe numérique, on choisit une clé de chiffrement. On regarde alors la position de chaque lettre de la clé dans l'alphabet, et on utilise cette position pour chiffrer de façon cyclique le message en clair. Exemple : si l'on souhaite chiffrer le message en clair `"QUOI"` avec la clé de chiffrement `"OUI"`, on commence par calculer les positions de la clé dans l'alphabet : on obtient respectivement 14, 20 et 8 (A étant un décalage de 0, et Z de 25). On décale ensuite chaque lettre en suivant ces valeurs de décalage cycliquement : le Q est décalé de 14, le U le 20, le O de 8, le I de 14. Cela donne le message chiffré `"EOWW"`.

---

1. Indication : vous pouvez utiliser la fonction `isalpha()` pour vérifier si un caractère est une lettre. Par exemple, si vous avez `c = 'a'`, alors `c.isalpha()` retourne `True` ; mais si `c = '!'` ou `c = '9'`, alors `c.isalpha()` retourne `False`.
2. Rappel : en Python, c'est l'opérateur arithmétique % qui fait le modulo : ainsi 30%26 donne 4.
3. `input` est une fonction qui prend en paramètre un message à afficher, et retourne une valeur précisée par l'utilisateur au moment de l'exécution du code Python. Par exemple, si dans votre Notebook vous exécutez la ligne `age = input("Quel est votre âge ?")`, le message `"Quel est votre âge ?"` s'affichera (comme avec un `print`), et une petite boîte apparaîtra où l'on peut rentrer une valeur. Une fois renseignée, celle-ci est stockée dans la variable `age`. Attention : la fonction `input` est sensible à la casse, il faut donc caster le type de la variable récupérée. Concrètement, il s'agit d'entourer l'appel à `input` de `int()` si vous voulez un entier, par exemple : `age = int(input("Quel est votre âge ?"))`
4. `https://fr.wikipedia.org/wiki/Chiffre_de_Vigenère`